

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Интеллектуальные системы в гуманитарной сфере**

Направление подготовки **45.03.04 Интеллектуальные системы в гуманитарной сфере**

Курсовая работа
по дисциплине «Мобильная разработка»

Тема: Разработка мобильного приложения «Мессенджер»

Обучающаяся: Шадрина Александра Валерьевна, МОБРАЗ 2.1

Работа выполнена с оценкой _____

Преподаватель:
Шатинский Григорий Сергеевич

(подпись)

Дата 20.01.2026

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Разработка базовой архитектуры и системы навигации приложения.....	5
1.1 Выбор архитектурного паттерна.....	5
1.2 Программная реализация и управление жизненным циклом.....	7
2 Управление состоянием, реализация паттерна MVVM.....	10
3 Работа с сетью и локальное хранение данных.....	14
3.1 Локальное хранение данных с использованием Room.....	14
3.2 Сетевое взаимодействие через Retrofit.....	15
3.3 Реализация паттерна Repository и логика кэширования.....	16
3.4 ViewModel, RecyclerView и Adapter.....	17
4 Фоновые процессы, уведомления и визуальное совершенствование интерфейса.....	18
4.1 Разработка пользовательского интерфейса.....	18
4.2 Фоновая синхронизация с использованием WorkManager.....	18
5 Разработанное решение.....	22
ЗАКЛЮЧЕНИЕ.....	27

ВВЕДЕНИЕ

В современной ИТ-индустрии мобильная разработка занимает одну из лидирующих позиций, поскольку смартфоны стали основным инструментом потребления контента и коммуникации. Создание качественного мессенджера требует от разработчика не только понимания визуальной составляющей, но и глубоких знаний в области архитектуры, асинхронного программирования и управления данными.

Актуальность данной работы обусловлена необходимостью изучения и практического применения современных подходов к разработке под операционную систему Android, которая на сегодняшний день является самой распространенной мобильной платформой в мире.

Основной целью курсового проекта является проектирование и реализация прототипа мобильного мессенджера с использованием наиболее актуального технологического стека. В процессе работы над проектом решается комплекс задач, связанных с построением гибкой и масштабируемой архитектуры приложения. Одним из ключевых аспектов является переход от простых монолитных решений к использованию паттерна проектирования MVVM, который позволяет разделить ответственность между компонентами и обеспечить тестируемость кода. Особое внимание уделяется пользовательскому опыту, который в современных условиях невозможен без быстрой навигации, интуитивно понятных элементов управления и поддержки различных режимов отображения, таких как темная тема.

Важной составляющей современной мобильной разработки является обеспечение надежности хранения и передачи данных. В рамках введения стоит отметить, что современное приложение не может полагаться исключительно на стабильное интернет-соединение. Пользователи ожидают, что приложение останется работоспособным даже в условиях отсутствия сети. Именно поэтому в данном проекте рассматривается концепция Offline-first, предполагающая наличие надежного локального кэша.

Использование реляционных баз данных внутри мобильного устройства и их синхронизация с внешними серверами через REST API являются критически важными навыками для инженера-разработчика. Эти технологии позволяют создавать продукты, которые эффективно используют ресурсы устройства и обеспечивают сохранность информации.

Кроме того, в работе исследуются механизмы фонового выполнения задач и уведомлений, которые играют роль связующего звена между приложением и пользователем. В условиях жесткой конкуренции на рынке мобильного ПО крайне важно удерживать внимание аудитории и предоставлять актуальную информацию в режиме реального времени. Изучение таких инструментов, как фоновые воркеры и менеджеры уведомлений, позволяет автоматизировать рутинные процессы обновления контента. В совокупности все рассматриваемые в курсовом проекте технологии и подходы формируют целостное представление о цикле разработки мобильного продукта: от проектирования структуры одного экрана до создания сложной системы, способной функционировать в фоновом режиме и эффективно взаимодействовать с облачными сервисами.

1 Разработка базовой архитектуры и системы навигации приложения

1.1 Выбор архитектурного паттерна

Для реализации проекта «Мессенджер» была выбрана современная архитектура Single Activity (одна активность – множество фрагментов). В отличие от подхода, где каждый экран является отдельной Activity, использование фрагментов под управлением одного хоста позволяет:

- снизить потребление ресурсов устройства при переключении между экранами,
- обеспечить плавную анимацию переходов,
- упростить организацию навигации с помощью единого графа,
- легко внедрять общие компоненты интерфейса (например, нижнюю панель меню), которые не пересоздаются при смене контента.

Центральной точкой входа является MainActivity. Ее разметка (activity_main.xml) разделена на две логические зоны (рис. 1).

```
<fragment
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:defaultNavHost="true"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toTopOf="@id/bottom_nav"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:navGraph="@navigation/nav_graph" />

<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottom_nav"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:background="@color/bg_bottom_color"
    android:elevation="8dp"
    app:itemIconTint="@color/bottom_nav_color"
    app:itemTextColor="@color/bottom_nav_color"
    app:menu="@menu/bottom_nav_menu"
    app:labelVisibilityMode="labeled"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Рисунок 1 – Макет activity_main.xml

Первая – контейнер NavHostFragment – окно, внутри которого происходит подмена фрагментов (экранов приложения). Параметр app:navGraph указывает системе, по какому сценарию будет происходить замена контента.

Вторая – панель BottomNavigationView – глобальный элемент навигации, позволяющий пользователю переключаться между основными разделами. Всего в приложении 3 раздела – Новости, Профиль и Настройки. Они определены в меню навигации в нижней части экрана, в файле bottom_nav_menu.xml (рис. 2)

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/newsFragment"
        android:icon="@drawable/ic_news"
        android:title="Новости" />

    <item
        android:id="@+id/profileFragment"
        android:icon="@drawable/ic_profile"
        android:title="Профиль" />

    <item
        android:id="@+id/settingsFragment"
        android:icon="@drawable/ic_settings"
        android:title="Настройки" />

</menu>
```

Рисунок 2 – Код bottom_nav_menu.xml

Вся логика переходов между экранами вынесена в отдельный ресурсный файл – nav_graph.xml. Это позволяет визуализировать структуру приложения и избежать «жесткого» написания кода для транзакций фрагментов. Код представлен на рисунке 3.

```

<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    app:startDestination="@id/newsFragment">

    <fragment
        android:id="@+id/newsFragment"
        android:name="com.example.messengerapp.NewsFragment"
        android:label="Новости" />

    <fragment
        android:id="@+id/profileFragment"
        android:name="com.example.messengerapp.ProfileFragment"
        android:label="Профиль" />

    <fragment
        android:id="@+id/settingsFragment"
        android:name="com.example.messengerapp.SettingsFragment"
        android:label="Настройки" />
</navigation>

```

Рисунок 3 – Код nav_graph.xml

Каждый экран зарегистрирован как <fragment> с уникальным id. Принципиально важно, чтобы id фрагмента в графе совпадал с id соответствующего пункта в bottom_nav_menu.xml. Это позволяет библиотеке Navigation Component автоматически связывать нажатие на кнопку меню с открытием нужного экрана.

1.2 Программная реализация и управление жизненным циклом

Логика управления приложением сосредоточена в классе MainActivity.kt. В методе onCreate выполняется инициализация NavController. Этот объект является «мозгом» навигации (рис. 4).

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Log.d(tag, msg = "onCreate called")
    setContentView(R.layout.activity_main)

    val navController = findNavController( viewId = R.id.nav_host_fragment)
    val bottomNav = findViewById<BottomNavigationView>( id = R.id.bottom_nav)
    bottomNav.setupWithNavController(navController)
}

```

Рисунок 4 – Фрагмент MainActivity

Метод `setupWithNavController` избавляет разработчика от написания слушателей нажатий (`onNavigationItemSelectedListener`). Библиотека сама понимает, какой фрагмент показать.

Для анализа стабильности приложения и понимания механизмов управления памятью в Android, были переопределены методы жизненного цикла Activity. В каждый метод добавлено логирование:

- `onCreate`: начальная инициализация, создание View;
- `onStart/onResume`: подготовка интерфейса к взаимодействию с пользователем;
- `onPause/onStop`: сохранение данных и приостановка тяжелых операций при сворачивании приложения;
- `onDestroy`: финальная очистка ресурсов перед уничтожением объекта активности.

Пример реализации логирования представлен на рисунке 5.

```

override fun onPause() {
    super.onPause()
    Log.d(tag, msg = "onPause called")
}

```

Рисунок 5 – Логирование onPause

Далее рассмотрим реализацию одного из экранов приложения. Экран настроек демонстрирует работу с постоянным хранилищем данных и динамическим изменением темы приложения. Для сохранения выбора пользователя (темная или светлая тема) использован механизм

SharedPreferences. Это позволяет настройкам сохраняться даже после полного закрытия приложения.

```
override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
    super.onCreateView(view, savedInstanceState)  
  
    val switchTheme = view.findViewById<SwitchMaterial>( id = R.id.switchTheme)  
    val prefs = requireContext().getSharedPreferences( p0 = "settings", p1 = Context.MODE_PRIVATE)
```

Рисунок 6 – Фрагмент кода SettingsFragment

Использование `AppCompatActivity.setDefaultNightMode` позволяет мгновенно изменить цветовую палитру приложения без необходимости перезагрузки текущей Activity вручную. Система сама перерисовывает компоненты в соответствии с выбранным режимом (`MODE_NIGHT_YES` или `MODE_NIGHT_NO`).

По итогам первой главы была создана масштабируемая архитектура мобильного мессенджера. Приложение корректно обрабатывает навигационные события, поддерживает сохранение пользовательских предпочтений и позволяет отслеживать системные события через механизмы логирования жизненного цикла.

2 Управление состоянием и реализация архитектурного паттерна MVVM

Следующим этапом работы была произведена миграция логики приложения на паттерн MVVM (Model-View-ViewModel). Основная цель этого перехода – разделение ответственности:

- View (Fragment) отвечает исключительно за отображение данных и взаимодействие с пользователем (нажатия, ввод текста);
- ViewModel хранит состояние экрана и обрабатывает бизнес-логику. Она не имеет ссылок на View, что предотвращает утечки памяти;
- LiveData выступает в роли «реактивного моста», позволяя View автоматически обновляться при изменении данных во ViewModel.

Одной из ключевых задач является сохранение данных при повороте экрана (Configuration Changes). Стандартный жизненный цикл Activity/Fragment приводит к их уничтожению и повторному созданию. Использование ViewModel решает эту проблему, так как объект ViewModel сохраняется в памяти до тех пор, пока экран не будет окончательно закрыт (выход из приложения или закрытие фрагмента).

Для каждого фрагмента была реализована своя ViewModel: для профиля и настроек – типовые; ViewModel для новостей будет рассмотрена позднее.

Класс ProfileViewModel инкапсулирует данные пользователя: имя и статус. Использование MutableLiveData позволяет реализовать реактивное обновление интерфейса. Код класса представлен на рисунке 7.

```

package com.example.messengerapp.viewmodel

import android.util.Log
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

2 Usages
class ProfileViewModel : ViewModel() {

    3 Usages
    val userName = MutableLiveData<String>( value = "Имя пользователя")
    3 Usages
    val status = MutableLiveData<String>( value = "Статус")

    init {
        Log.d( tag = "ProfileViewModel", msg = "ViewModel created")
    }

    override fun onCleared() {
        Log.d( tag = "ProfileViewModel", msg = "ViewModel cleared")
        super.onCleared()
    }
}

```

Рисунок 7 – ProfileViewModel

В ProfileFragment реализована подписка (Observer) на данные из ViewModel. Как только значение во ViewModel меняется (например, после редактирования в диалоговом окне), UI обновляется автоматически. Для редактирования данных использовано диалоговое окно AlertDialog. При нажатии кнопки «Сохранить», данные из полей ввода EditText передаются напрямую во ViewModel, что мгновенно отражается на основном экране профиля. Фрагмент основного кода ProfileFragment представлен на рисунке 8.

```

profileViewModel.userName.observe(viewLifecycleOwner) { name ->
    usernameTextView.text = name
}

profileViewModel.status.observe(viewLifecycleOwner) { status ->
    aboutTextView.text = status
}

editProfileButton.setOnClickListener {
    val dialogView = LayoutInflater.inflate( resource = R.layout.dialog_edit_profile, root = null)

    val editName = dialogView.findViewById<EditText>( id = R.id.editUserName)
    val editStatus = dialogView.findViewById<EditText>( id = R.id.editStatus)

    editName.setText(profileViewModel.userName.value)
    editStatus.setText(profileViewModel.status.value)

    val alertDialog = AlertDialog.Builder( context = requireContext())
        .setTitle("Редактирование профиля")
        .setView(dialogView)
        .setPositiveButton( text = "Сохранить") { _, _ ->
            profileViewModel.userName.value = editName.text.toString()
            profileViewModel.status.value = editStatus.text.toString()
        }
        .setNegativeButton( text = "Отмена", listener = null)
        .create()

    editName.requestFocus()
    alertDialog.setOnShowListener {

```

Рисунок 8 – ProfileFragment

В экране настроек (SettingsFragment) ViewModel используется для хранения состояния темы (светлая/темная). Логика реализации представлена на рисунке 9.

```

settingsViewModel.isDarkTheme.observe(viewLifecycleOwner) { isDark ->
    ignoreListener = true
    if (switchTheme.isChecked != isDark) {
        switchTheme.isChecked = isDark
    }
    ignoreListener = false
}

switchTheme.setOnCheckedChangeListener { _, isChecked ->
    if (ignoreListener) return@setOnCheckedChangeListener

    settingsViewModel.isDarkTheme.value = isChecked
    prefs.edit { putBoolean( p0 = "dark_theme", p1 = isChecked) }

    AppCompatDelegate.setDefaultNightMode(
        if (isChecked) AppCompatDelegate.MODE_NIGHT_YES
        else AppCompatDelegate.MODE_NIGHT_NO
    )
}

```

Рисунок 9 – SettingsFragment

Важной деталью реализации является переменная `ignoreListener`. Она предотвращает «зацикливание» при программной установке состояния переключателя во время срабатывания наблюдателя `LiveData`.

Согласно функциональным требованиям, в коде реализовано отслеживание жизненного цикла `ViewModel`. Инициализация (`init`) фиксируется в логах при первом обращении к `ViewModel` (например, при открытии фрагмента). Завершение (`onCleared`) вызывается системой, когда фрагмент окончательно удаляется из стека (например, при выходе из приложения), что позволяет очистить ресурсы. Это гарантирует, что данные пользователя (введенные в профиле или настройки темы) остаются консистентными на протяжении всей сессии работы с приложением, включая повороты экрана и временное сворачивание.

Внедрение архитектуры `MVVM` позволило создать надежное приложение с четким разделением логики и интерфейса. Использование `LiveData` обеспечило реактивность UI, а `ViewModel` решила задачу сохранения данных при изменении конфигурации устройства.

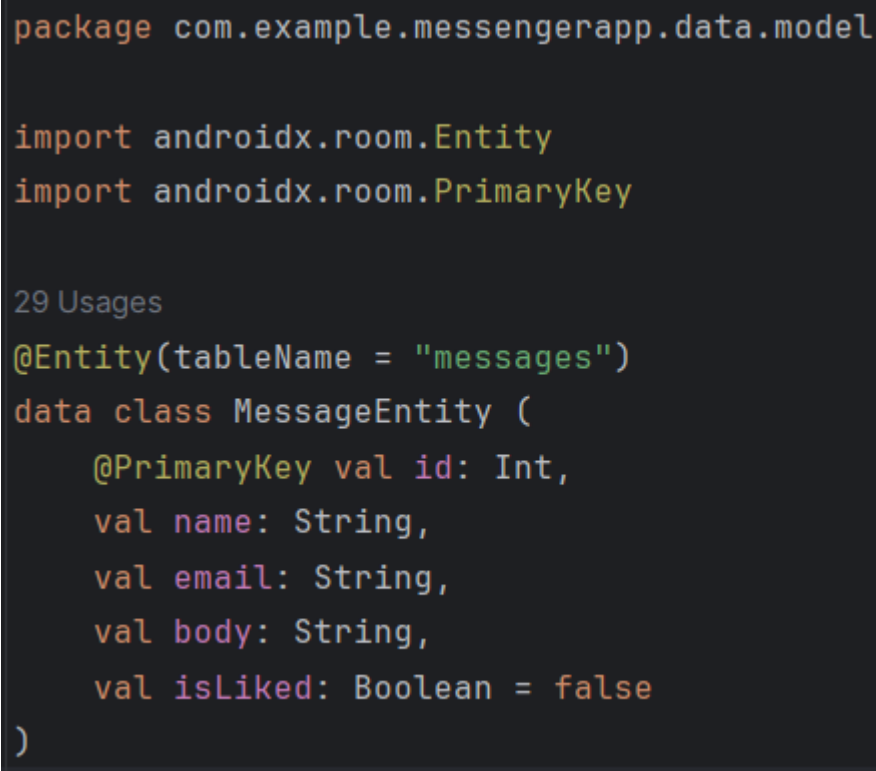
3 Работа с сетью и локальное хранение данных

Целью следующего этапа стала реализация надежного механизма получения и хранения данных. В приложение была внедрена стратегия Offline-first, которая гарантирует доступность контента даже при отсутствии интернет-соединения. Для этого был использован паттерн «Репозиторий», который выступает единой точкой доступа к данным, скрывая от вызывающего кода (ViewModel) детали реализации запросов к API или базе данных.

3.1 Локальное хранение данных с использованием Room

Для кэширования сообщений использована библиотека Room. Это объектно-реляционная надстройка над SQLite, которая позволяет работать с БД на уровне типизированных объектов Kotlin.

Так, MessageEntity описывает структуру таблицы сообщения, которые будут высвечиваться на экране новостей. Код представлен на рисунке 10.



```
package com.example.messengerapp.data.model

import androidx.room.Entity
import androidx.room.PrimaryKey

29 Usages
@Entity(tableName = "messages")
data class MessageEntity (
    @PrimaryKey val id: Int,
    val name: String,
    val email: String,
    val body: String,
    val isLiked: Boolean = false
)
```

Рисунок 10 – Структура таблицы Сообщения

MessageDao содержит методы для взаимодействия с БД. Метод insertAll использует стратегию REPLACE, что автоматически обновляет данные при совпадении ID. Код представлен на рисунке 11.

```
package com.example.messengerapp.data.local

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.example.messengerapp.data.model.MessageEntity

7 Usages 1 Implementation
@Dao
interface MessageDao {
    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM messages")
    suspend fun getAll(): List<MessageEntity>

    2 Usages 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages: List<MessageEntity>)

    1 Usage 1 Implementation
    @Query(value = "DELETE FROM messages")
    suspend fun clear()
}
```

Рисунок 11 – MessageDao

3.2 Сетевое взаимодействие через Retrofit

Для получения списка сообщений использован HTTP-клиент Retrofit. Запросы выполняются к тестовому API (jsonplaceholder).

ApiService (рис. 12) описывает эндпоинты. Использование модификатора suspend позволяет выполнять сетевые вызовы в рамках корутин, не блокируя основной поток (UI-thread).

```
package com.example.messengerapp.data.remote

import retrofit2.http.GET
import com.example.messengerapp.data.model.MessageEntity

4 Usages
interface ApiService {
    1 Usage
    @GET(value = "comments")
    suspend fun getMessages(): List<MessageEntity>
}
```

Рисунок 12 – ApiService

RetrofitClient (рис. 13) – синглтон-объект, настраивающий базовый URL и конвертер GsonConverterFactory для автоматического парсинга JSON в объекты MessageEntity.

```
package com.example.messengerapp.data.remote

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

4 Usages
object RetrofitClient {

    1 Usage
    private const val BASE_URL = "https://jsonplaceholder.typicode.com/"

    val api: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl( baseUrl = BASE_URL)
            .addConverterFactory( factory = GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}
```

Рисунок 13 – RetrofitClient

3.3 Реализация паттерна Repository и логика кэширования

Класс MessageRepository координирует работу двух источников данных. Основная логика метода getMessages() заключается в попытке обновить данные из сети:

- при вызове приложение обращается к API,
- в случае успеха локальная база очищается (dao.clear()), и новые данные записываются в Room,
- в случае ошибки (например, нет интернета) блок catch перехватывает исключение и возвращает данные из локальной базы.

Логика представлена на рисунке 14.


```

suspend fun getMessages(): List<MessageEntity> {
    return try {
        val messages = api.getMessages()
        dao.clear()
        dao.insertAll(messages)
        messages
    } catch (e: Exception) {
        dao.getAll()
    }
}

```

Рисунок 14 – Логика загрузки сообщений

3.4 ViewModel, RecyclerView и Adapter

FeedViewModel управляет процессом загрузки. Все вызовы производятся внутри viewModelScope.launch. Для визуализации ленты новостей использован RecyclerView с пользовательским адаптером MessagesAdapter, который реализует паттерн ViewHolder для эффективной перерисовки элементов. Метод submitList обновляет содержимое списка.

В итоге Retrofit скачивает данные, Repository сохраняет их в Room, ViewModel запускает этот процесс в фоновой корутине и выставляет результат в LiveData. Fragment замечает изменения и просит Adapter обновить RecyclerView.

Реализованная система обеспечивает стабильную работу приложения в условиях нестабильного интернет-соединения. Связка Room + Retrofit + Coroutines является надежным фундаментом для любого современного мобильного приложения, обеспечивая высокую скорость отклика интерфейса и сохранность данных.

4 Фоновые процессы, уведомления и визуальное совершенствование интерфейса

4.1 Разработка пользовательского интерфейса

Интерфейс приложения был приведен к стандартам Material Design 3. Основное внимание уделено карточной системе отображения контента.

Компоненты MaterialDesign:

- CardView использован для каждого сообщения в ленте. Это позволило визуально отделить элементы друг от друга с помощью теней и скругления углов (`app:cardCornerRadius="12dp"`), создавая эффект глубины;
- В FloatingActionButton вынесена основная функция обновления. Использование FAB подчеркивает важность действия и делает его доступным в один клик;
- AppBarLayout и Toolbar обеспечивают стандартное поведение верхней панели с поддержкой темной темы и корректным отображением заголовков.

Элемент списка (`item_message.xml`) ленты новостей был расширен. Теперь он включает аватарку пользователя, структурированные текстовые поля и интерактивную иконку «лайка». Логика «лайка» реализована через callback-функцию: при нажатии на иконку во View, событие пробрасывается во ViewModel, меняет состояние в LiveData, и адаптер мгновенно перерисовывает иконку (с контурной на закрашенную), обеспечивая быстрый визуальный отклик.

4.2 Фоновая синхронизация с использованием WorkManager

Для обеспечения актуальности данных в мессенджере внедрена система фоновой синхронизации. Инструмент WorkManager был выбран как наиболее надежный способ выполнения отложенных задач, гарантирующий исполнение даже после перезагрузки устройства.

Также был создан класс SyncMessagesWorker, наследуемый от CoroutineWorker. Это позволяет использовать все преимущества корутин (асинхронность) внутри фоновой задачи. Воркер самостоятельно получает

экземпляр базы данных, создает репозиторий, скачивает данные из API и обновляет локальный кэш Room. При успешном завершении вызывается метод отправки уведомления. Код воркера представлен на рисунке 15.

```
class SyncMessagesWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker( applicationContext = context, params) {

    override suspend fun doWork(): Result {
        return try {
            val database = AppDatabase.getDatabase(applicationContext)
            val repository = MessageRepository(
                api = RetrofitClient.api,
                dao = database.messageDao()
            )

            val messages = repository.getMessages()
            repository.saveMessages(messages)
            NotificationHelper.showSyncNotification(applicationContext)
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}
```

Рисунок 15 – SyncMessagesWorker

В MainActivity был настроен периодический запрос (PeriodicWorkRequest) с интервалом в 15 минут. Использование ExistingPeriodicWorkPolicy.KEEP предотвращает создание дубликатов задач при каждом перезапуске приложения. Фрагмент кода представлен на рисунке 16.

```

val workRequest =
    PeriodicWorkRequestBuilder<SyncMessagesWorker>(
        repeatInterval = 15, repeatIntervalTimeUnit = TimeUnit.MINUTES
    ).build()

WorkManager.getInstance(context = this).enqueueUniquePeriodicWork(
    uniqueWorkName = "sync_messages",
    ExistingPeriodicWorkPolicy.KEEP,
    periodicWork = workRequest
)

NotificationUtils.createNotificationChannel(context = this)
requestNotificationPermission()

```

Рисунок 16 – Фрагмент кода MainActivity

Для информирования пользователя о завершении синхронизации разработана система уведомлений. Начиная с Android 8.0 (API 26), уведомления требуют наличия каналов. В классе NotificationUtils реализовано создание канала с высоким приоритетом (IMPORTANCE_HIGH), что позволяет пользователю гибко настраивать важность уведомлений приложения в системных настройках. Класс NotificationHelper инкапсулирует логику сборки уведомления через NotificationCompat.Builder. Уведомление содержит иконку, заголовок и текст «Новые данные получены».

В соответствии с требованиями безопасности Android 13 (API 33) и выше, приложение реализует динамический запрос разрешений на отправку уведомлений. В MainActivity реализован метод requestNotificationPermission() – приложение проверяет наличие разрешения Manifest.permission.POST_NOTIFICATIONS; если разрешение не дано, приложение предлагает пользователю подтвердить его через системный диалог, что является обязательным условием для работы пуш-уведомлений в современных версиях ОС.

Итак, реализация WorkManager и системы уведомлений позволила мессенджеру работать автономно, а использование Material Design обеспечило современный и интуитивно понятный интерфейс.

5 Разработанное решение

В результате выполнения четырех этапов проектирования было создано полнофункциональное приложение «Мессенджер», представляющее собой многоуровневую систему с четким распределением зон ответственности. Итоговая структура проекта представлена на рисунке 17.

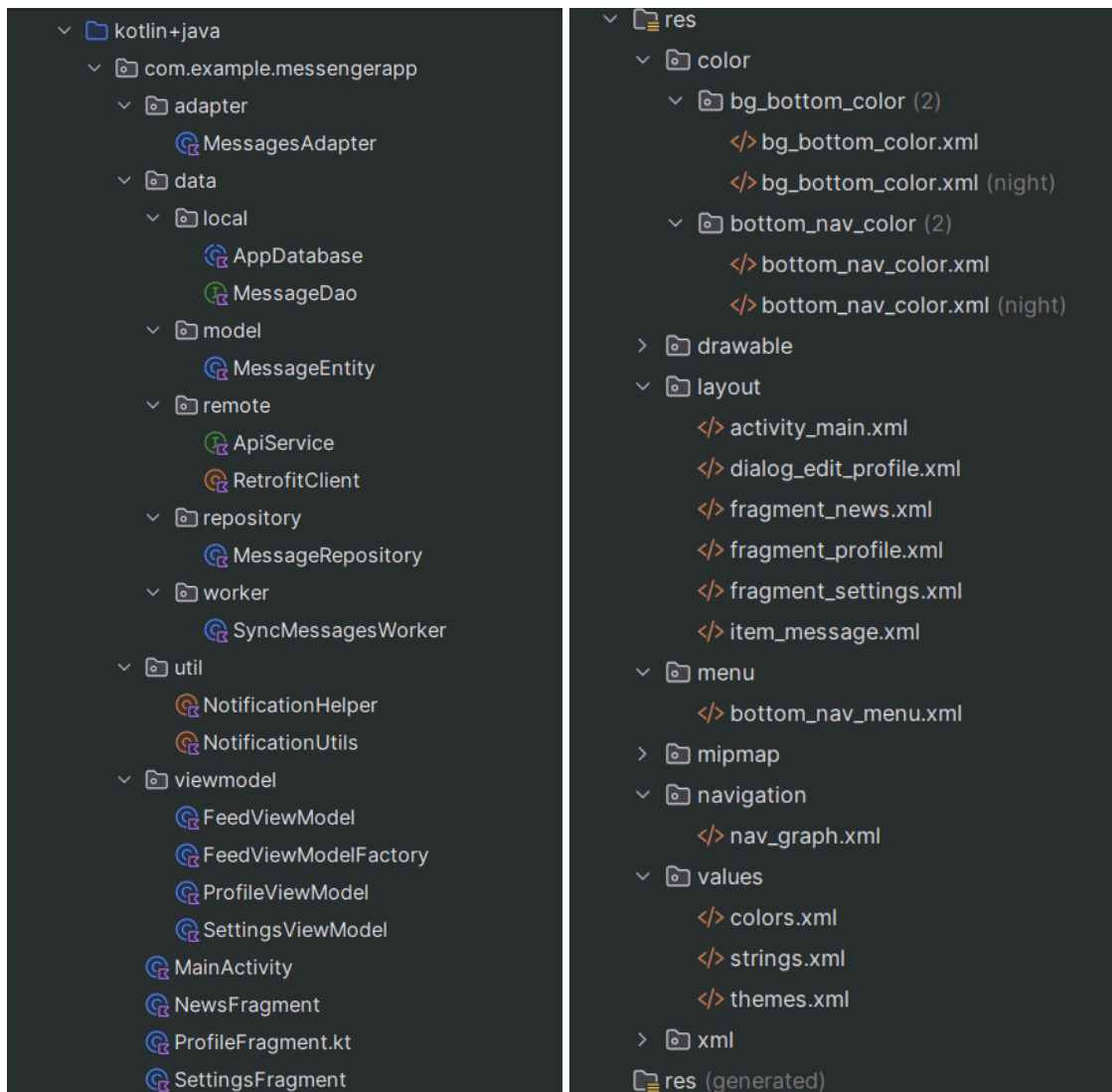


Рисунок 17 – Структура проекта

В основе приложения лежит архитектурная концепция Single Activity, где единственная активность выступает в роли глобального контейнера и координатора навигации. Переходы между ключевыми экранами осуществляются декларативно через граф навигации, что исключает ошибки прямого манипулирования фрагментами и делает структуру переходов прозрачной. Визуальная часть приложения построена с использованием

компонентов Material Design, что обеспечивает эстетичный и интуитивно понятный пользовательский опыт. Экраны профиля, новостной ленты и настроек логически разделены и поддерживают единый стиль оформления, включая адаптацию под светлую и темную темы оформления за счет интеграции с системными настройками Android.

Фундаментальной особенностью проекта является использование паттерна проектирования MVVM. Это позволило полностью отделить бизнес-логику и управление данными от пользовательского интерфейса. Хранение состояния экранов во ViewModel гарантирует стабильность работы приложения при изменении конфигурации устройства, например, при повороте экрана, когда данные не теряются и не запрашиваются повторно из сети. Реактивность системы обеспечена применением LiveData, благодаря чему любое изменение в слое данных мгновенно отражается на экране без необходимости принудительного обновления интерфейса пользователем.

Слой данных в приложении реализован через паттерн «Репозиторий», который объединяет работу с внешним API и локальным хранилищем Room. Это наделило мессенджер полноценным офлайн-режимом: при отсутствии интернет-соединения пользователь всегда видит актуальный кэш сообщений, сохраненный в базе данных SQLite. Сетевое взаимодействие через Retrofit и операции с базой данных выполняются асинхронно с помощью корутин, что исключает блокировку графического потока и обеспечивает высокую плавность работы интерфейса даже при обработке больших массивов информации в списках RecyclerView.

Завершающим элементом системы стала автоматизация фоновых процессов. Внедрение WorkManager позволило реализовать периодическую синхронизацию данных в фоновом режиме, что делает приложение автономным и всегда актуальным. Система уведомлений, работающая в связке с фоновыми задачами, своевременно информирует пользователя о получении новых данных, создавая ощущение активного сервиса. Безопасность и соответствие современным требованиям операционной

системы подтверждаются реализованным механизмом запроса разрешений в реальном времени. Таким образом, итоговый программный продукт представляет собой масштабируемое, отказоустойчивое и удобное приложение, готовое к дальнейшему расширению функционала.

Ниже представлены скриншоты, демонстрирующие работоспособность описанных функций и корректность отображения элементов интерфейса в различных режимах эксплуатации.

Так, экран ленты содержит список сообщений (новостей), для каждого указаны аватарка, имя пользователя, email и текст. Также есть возможность поставить лайк сообщению. При нажатии на кнопку обновления приложение уведомляет пользователя об успешной синхронизации. Экран ленты в двух версиях представлен на рисунке 18.

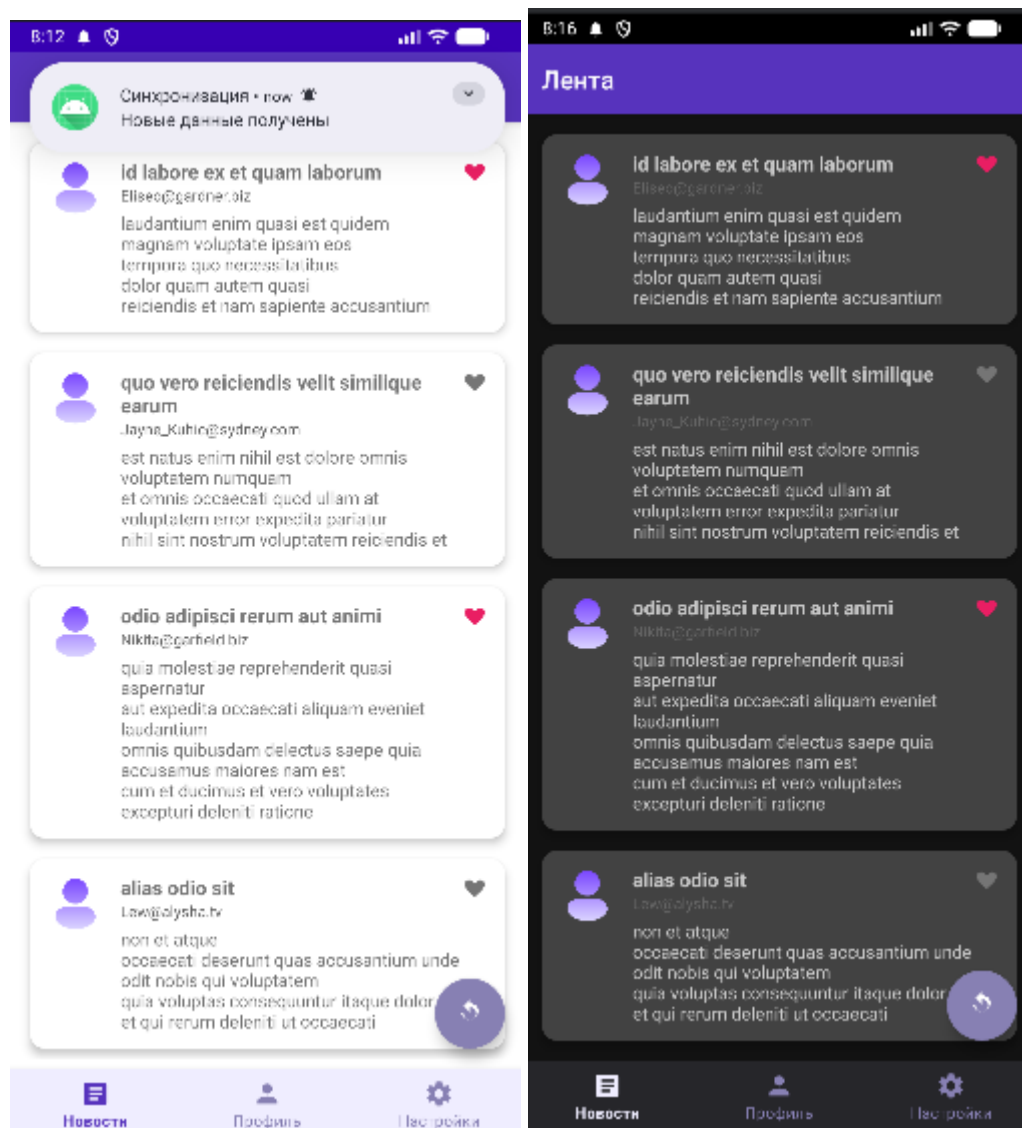


Рисунок 18 – Экран ленты новостей

Экран профиля содержит основную информацию о пользователе. Поддерживается возможность редактирования профиля в отдельном диалоговом окне – обновление имени пользователя и статуса. Экран профиля представлен на рисунке 19.

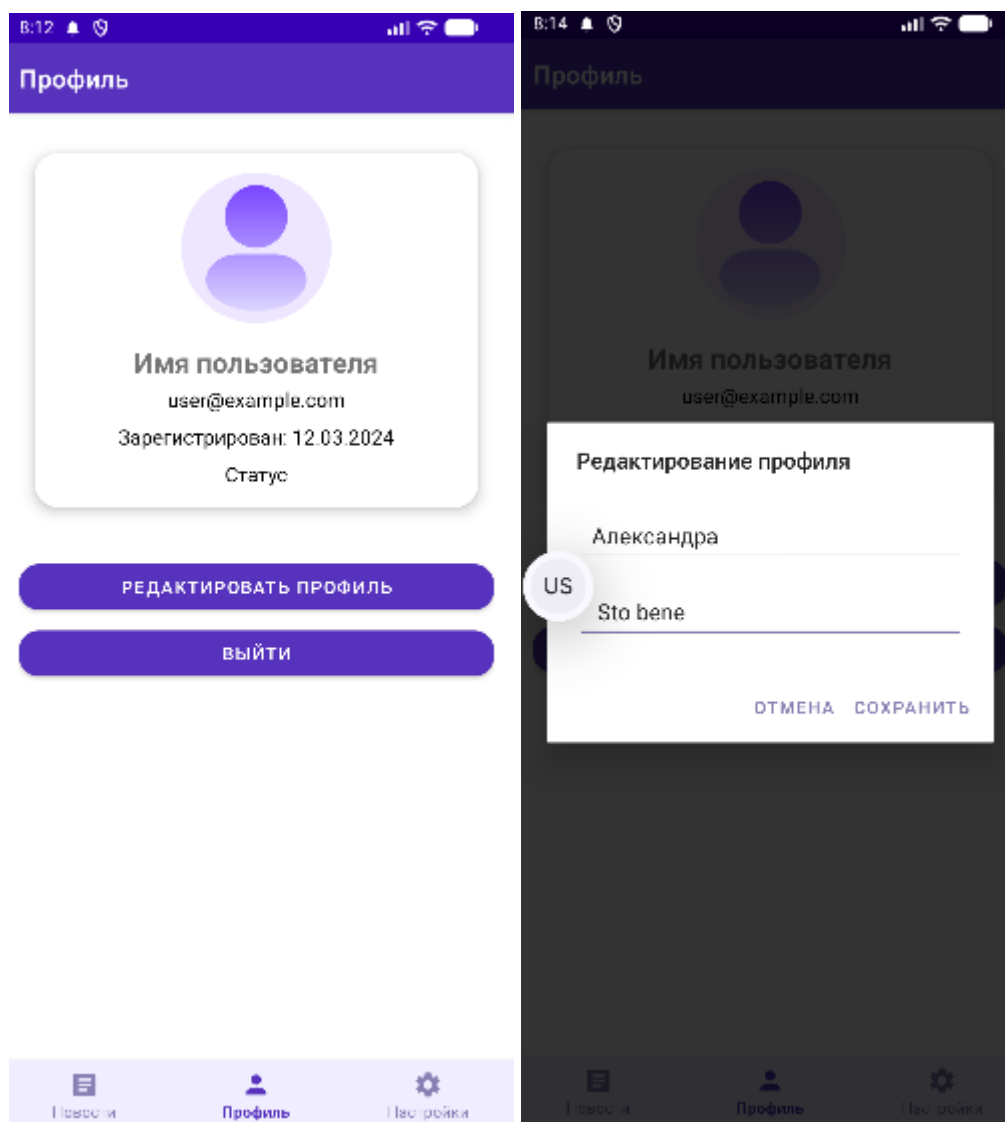


Рисунок 19 – Экран профиля

Экран настроек содержит единственно тумблер переключения темы со светлой на темную и представлен на рисунке 20.

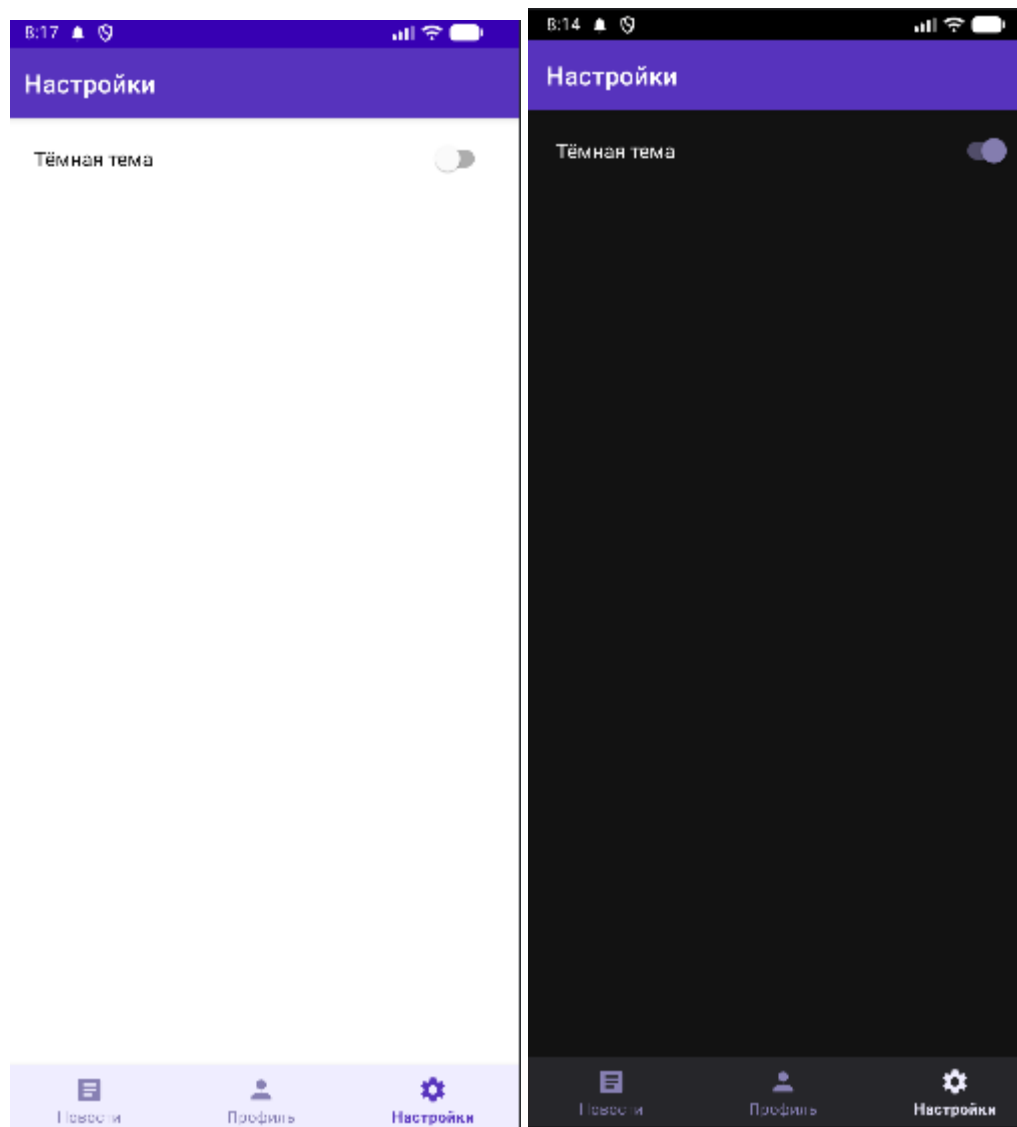


Рисунок 20 – Экран настроек

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была проведена комплексная работа по проектированию и реализации мобильного приложения «Мессенджер» для платформы Android. Основным итогом работы стало создание программного продукта, который не только выполняет заявленные функциональные требования, но и соответствует современным индустриальным стандартам архитектурного проектирования. В процессе разработки удалось успешно решить задачи по организации навигации, управлению состоянием пользовательского интерфейса, а также по настройке стабильного взаимодействия с внешними и внутренними источниками данных.

Особое значение в работе имело практическое освоение архитектурного паттерна MVVM и реактивного программирования. Реализованное разделение логики на уровни представления, бизнес-логики и данных позволило создать отказоустойчивую систему, способную сохранять целостность при изменении конфигурации устройства. Применение современных библиотек, таких как Room и Retrofit, в связке с механизмом корутин обеспечило высокую производительность приложения и плавность интерфейса, что является критически важным фактором для пользовательского опыта в сегменте мобильных коммуникаций.

Кроме того, в проекте была успешно реализована стратегия автономной работы приложения. Создание надежного репозитория и механизмов фоновой синхронизации данных через WorkManager позволило решить проблему доступности контента в условиях нестабильного сетевого соединения. Интеграция системы уведомлений и динамического управления разрешениями обеспечила интерактивность приложения и его соответствие актуальным требованиям безопасности операционной системы Android. Это подтверждает, что заложенная в проекте база является масштабируемой и пригодной для дальнейшего развития, например, для внедрения реального чата или системы авторизации.

Подводя общий итог, можно утверждать, что все цели, поставленные в начале проектирования, были полностью достигнуты. Полученный опыт разработки охватил полный цикл создания мобильного сервиса: от формирования структуры макетов до автоматизации фоновых процессов. Разработанное приложение «Мессенджер» представляет собой прототип, демонстрирующий эффективное использование технологического стека Kotlin и Java, что подтверждает готовность программного решения к эксплуатации и потенциальному расширению функциональных возможностей.